Aknakereső

Programozói dokumentáció:

Az aknakereső játékban három részre van osztva a funkciók kezelése. Az első rész a pálya létrehozásáért, esetleg betöltéséért és egyéb a „kulisszák mögött” történő eseményért felelős. Ehhez tartozó fájlok a palya.c és a palya.h-ban találhatóak.

palya.h:

```
typedef enum Mezo {
    felderitetlen_ures,
    felderitetlen_akna,
    felderitett_ures,
    akna,
    zaszlos_ures,
    zaszlos_akna,
    egy,
    ketto,
    harom,
    negy,
    ot,
    hat,
    het,
    nyolc,
} Mezo;
```

```
typedef struct Palya{
    Mezo** palya;
    int magassag;
    int szelesseg;
    int aknak;
} Palya;

typedef struct Kord{
    int x,y;
}Kord;
```

A forrásfájl elején definiálunk egy típust (Mezo), mely használatával könnyebben átláthatóvá válik, hogy mely mezőn milyen játékelem található.

A pálya létrehozásához létrehozunk egy struktúrát, melyben az adott pályára vonatkozó adatok tárolhatóak el. Fontos, hogy tudjuk a pályának az elemeit, melyet egy kétdimenziós tömbben fogunk eltárolni, a pálya szélességét, és magasságát, illetve hogy hány akna található a pályán.

Emellett még létrehozunk egy koordináta típust is, mely a pálya bejárását fogja segíteni a további feladatok során.

A fájl alján megnevezzük a palya.c-ben megfogalmazott függvényeket.

palya.c:

static bool uj(void); Ebben a függvényben derítjük ki a felhasználótól, hogy új játékot akar kezdeni, avagy a régit folytatni. Visszatérési értéke a válasz (egy bool típus).

static void akna_randomizalas(Palya *p); Ebben a függvényben fogjuk majd elhelyezni az aknákat randomizálva, hogy minden új játék során más helyre kerüljenek. Nincs visszatérési értéke, hiszen a címszerint átadott pálya elemeit fogja változtatni.

bool palyafugv(Palya *p); Ebben a függvényben fogjuk létrehozni a pályát. Ez az „uj” függvény segítségével lesz eldöntve, hogy új játékot kezdünk, avagy egy régit folytatunk. Mindkét esetben dinamikusan foglalunk helyet a Palya típusú változónk kétdimenziós tömbjének, illetve az adatokat beolvassuk (vagy a képernyőről, vagy a „régi.txt” fájlból). Ha új játékot kezdtünk, akkor a listát felderítetlen mezőkkel töltjük fel, ha régit folytatunk akkor azokat töltjük be az adott helyre. Az aknák randomizálása függvény, csak az új játék esetén kell hogy lefusson. A visszatérési érték bool típusú, ezzel lehet majd vizsgálni hogy valami gond történt-e a pálya létrehozása során.

bool mentes(Palya *p); Ezzel a függvénnyel lehet fájlba menteni egy félbehagyott játékot. A játék mentése a „régi.txt” fájlba történik. Visszatérési értéke bool, ezzel lehet vizsgálni, hogy sikeres volt-e a játék mentése.

A második funkciót végrehajtó páros a vezerles.c és a vezerles.h. Ezek az econio segítségével fogják működtetni a képrenyőn végrehajtandó műveleteket.

vezerles.h:

void vezerles(Palya *p); Ezt a függvényt akarjuk használni, a mainben, hisz ez fogja a vezérlést a játék végéig elvégezni. A működését a vezerles.c-ben fejtem ki.

vezerles.c:

static int maximalis(int a, int b);
static int minimalis(int a, int b);
Ezek a függvények segítenek abban, hogy ne indexeljük túl a tömböt majd léptetésnél, „ne lépjünk le a pályáról”.

static int kisrobbantas(Palya *p,int a, int b); Ez a függvény az adott mező körül vizsgálja meg hány akna található, visszatérési értéke a darabszám.

static bool robbantas(Palya *p,int a, int b); A „kisrobbantas” függvény segítségével dönti el, hogy mik legyenek az új értékei a kattintást körölvevő mezőinek. Visszatérési értéke bool, ennek segítségével tudjuk majd kezelni, ha aknába lépnénk.

void zaszlo(Palya *p,Kord hely); A zászlózás funkciót végrehajtó függvény.

void vezerles(Palya *p); A vezérlés econio segítségével történően megadja nekünk a játékos által végzett mozdulatokat, legyen az bármilyen figyelendő gomblenyomás. Ennek megfelelően hívja meg a fent említett függvényeket, és végzi el a kellő műveleteket. A pálya adott elemére lépést vizsgálja az elején, Enter esetén a robbantást, Tab esetén a zászlózást, és Escape esetén a fileba mentést hajtja véghez.

A harmadik funkció a játékos számára való megjelenítés, ebben a megjelenites.h és megjelenites.c segít.

megjelenites.h:

void kirajz(Palya *p,Kord hely); Ezzel fogjuk kirajzoltatni az pálya adott elemét.

void kezdeti(Palya *p); A játék kezdetekor meg kell jeleníteni a pályát, ebben segít eme függvény.

megjelenites.c:

void kirajz(Palya *p,Kord hely); Ez a fájl végzi el a pálya adott elemének kirajzolását, annak értékének függvényében.

void kirajz(Palya *p,Kord hely); Töröljük a képernyőt, majd kirajzoljuk a létrehozott pályát (avagy előzőleg betöltöttet, ezzel a függvénynek már nem kell foglalkoznia).

Mindezek meghívását a main.c-ben tesszük meg, melyben a program lefutása után felszabadítjuk a dinamikusan lefoglalt területet.

Felhasználói dokumentáció:

Az aknakereső játékban a játékos célja az, hogy felderítse az összes olyan mezőt, amin nem akna van, a lehető legkevesebb időn belül.

A játék elején lehetőség van választani, hogy egy előző játékot szeretne folytatni a játékos, avagy egy újat kezdeni. Az utóbbi esetén meg kell adnia mekkora pályán akar játszani (szélesség magasság formátumban), illetve azt, hogy hány aknát szeretne a pályára rakni.

```
Uj jatekot szeretnel kezdeni? (I/N) i
Mekkora legyen a t◆bla? "szelesseg magassag" form◆tumban: 4 4
Mennyi akna legyen a p◆ly◆n? 4
```

Ezek után megjelenik a játéktér, melynek a vezérlése a következő:

- felfele lépés: felfele nyíl
- lefele lépés: lefele nyíl
- jobbra lépés: jobbra nyíl
- balra lépés: balra nyíl
- adott mezőre tippelés: ENTER
- adott mező zászlózása: TAB
- a játék mentése, abbahagyása: ESCAPE

